Mira_DeepEval_Colab.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Mira v2 — DeepEval evaluation (baseline T1–T12)

This notebook sends the 12 baseline test prompts to **Mira's n8n webhook**, scores each answer with **DeepEval** (judge model: `gpt-4o-mini`), then attaches the scores to the matching **Langfuse** traces.

Metric	What it checks	Applies to
Faithfulness	Every claim in the answer is supported by the facts Mira gave its agent (`retrieval_context`) — i.e. no hallucination	Specialist answers (not clarifications)
Answer Relevancy	The answer addresses the user's question	All 12
Requirement Compliance (GEval)	The answer meets the problem statement's *Must contain / Must NOT* rules for that test	All 12

Note: Before running make sure of the following: (1) the Mira workflow is **Active** in n8n, (2) add Colab secrets `OPENAI_API_KEY`, `LANGFUSE_PUBLIC_KEY`, `LANGFUSE_SECRET_KEY` (left sidebar → key icon → enable *Notebook access*). Then *Runtime → Run all*.

[14] ✓ 12s

```
!pip install -q deepeval==4.2.8 pandas requests
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.5/1.5 MB 25.0 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 110.5/110.5 kB 10.8 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 858.7/858.7 kB 60.8 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 69.4/69.4 kB 6.9 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 129.6/129.6 kB 13.5 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 74.2 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 46.4/46.4 kB 3.8 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 40.7/40.7 kB 3.4 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
huggingface-hub 1.33.0 requires click<9.0.0,>=8.4.2, but you have click 8.3.3 which is incompatible.
```

1 · Configuration

[9] ✓ 4s

```
import os, time, json, requests, pandas as pd

MIRA_WEBHOOK_URL = "https://agentic145.app.n8n.cloud/webhook/mira"   # n8n → Webhook node → Production URL
LANGFUSE_HOST    = "https://cloud.langfuse.com"                      # EU region
JUDGE_MODEL      = "gpt-4o-mini"                                     # DeepEval judge
THRESHOLD        = 0.7                                               # pass mark for each metric (0–1)
PUSH_TO_LANGFUSE = True

def secret(name, prefix=None):
    # Read a key from Colab Secrets (falls back to env var / prompt). Never prints the value.
    val = None
    try:
        from google.colab import userdata
        val = userdata.get(name)
    except Exception as e:
        print(f"  {name}: not read from Colab Secrets ({type(e).__name__}) — will ask instead")
    if isinstance(val, dict):   # some Colab versions return a metadata dict instead of the text
        val = val.get("value") or val.get("secret") or next(
            (v for k, v in val.items() if isinstance(v, str) and v.strip() and v != name and k not in ("name", "key")), None)
    if not val:
        val = os.environ.get(name)
    if not val:
        from getpass import getpass
        val = getpass(f"{name}: ")
    val = str(val).strip()
    if prefix and not val.startswith(prefix):
        print(f"  ⚠️ {name} does not start with '{prefix}' — check you pasted the right key")
    return val

os.environ["OPENAI_API_KEY"] = secret("OPENAI_API_KEY", "sk-")
os.environ["DEEPEVAL_TELEMETRY_OPT_OUT"] = "YES"
LF_AUTH = (secret("LANGFUSE_PUBLIC_KEY", "pk-lf-"), secret("LANGFUSE_SECRET_KEY", "sk-lf-")) if PUSH_TO_LANGFUSE else None
print("Config OK — judge:", JUDGE_MODEL, "| Mira:", MIRA_WEBHOOK_URL)
```

```
Config OK — judge: gpt-4o-mini | Mira: https://agentic145.app.n8n.cloud/webhook/mira
```

2 · Baseline test set

`expected` restates the problem statement's *Expected Output Must Contain* column, plus the ground-truth facts the judge needs to check it (e.g. the real task counts). The judge treats it as a rubric, not literal text.

[10] ✓ 0s

```
TESTS = [
  ("T1", "Generate a project plan for the AI Adoption Project at ABCDE Ltd. Use the provided project description.",
   "MUST include phases, milestones and a timeline, grounded in the project description: 8 phases from Project Initiation to Monitoring & Review over 24 weeks, goals of reducing delivery delays by 15%, lowering operational costs by 10% and improving service responsiveness. MUST NOT invent goals, KPIs or buc
  ("T2", "Generate a project plan for: "We want to build a chatbot."",
   "MUST flag the request as insufficient detail and ask for scope, timeline and team size. MUST NOT generate a full project plan from one sentence."),
  ("T3", "Generate a risk assessment for the AI Adoption Project at ABCDE Ltd. Use the provided project description.",
   "MUST produce categorized risks relevant to a logistics AI adoption project (the register has 10: data quality, integration, talent, change resistance, ethical/legal, cost overruns, unrealistic expectations, model drift, security, vendor lock-in) with impact and mitigation. MUST NOT include generic risks
  ("T4", "Generate a risk assessment for: "New project starting soon."",
   "MUST flag the request as insufficient and ask for project details. MUST NOT invent project-specific risks from nothing."),
  ("T5", "Generate a weekly status report using the sample task board data for Sprint 3.",
   "MUST show tasks by status and reference the actual Sprint 3 tasks: T007 Shortlist AI use cases (In Progress), T025 Legal review of data usage (In Progress), T008 Define success criteria for each use case (To Do), T009 Vendor evaluation for ML platform (To Do). MUST NOT invent tasks."),
  ("T6", "Generate a weekly status report for: "Things are going fine."",
   "MUST state that no task data is available. MUST NOT fabricate task statuses or progress percentages."),
  ("T7", "What are the top 3 risks for the ABCDE Ltd project based on the risk data?",
   "MUST name three risks that exist in the project risk register (IDs R01-R10) and explain the basis for the ranking. MUST NOT invent risks that are not in the register."),
  ("T8", "Which tasks are blocked or at risk of missing their deadline based on the task board?",
   "MUST identify T024 Security review of AI infrastructure as BLOCKED, and check due dates against progress (e.g. T006 is overdue; T007 and T025 are due soon). MUST NOT invent tasks."),
  ("T9", "Generate a project plan for a 2-week project with no details provided.",
   "MUST ask for scope, goals and deliverables. MUST NOT generate a detailed plan from the timeline alone."),
  ("T10", "Summarize the current project status: how many tasks are done, in progress, and to do?",
   "MUST give counts that match the task board exactly: Done 5, In Progress 3, Blocked 1, To Do 16, Total 25."),
  ("T11", "What milestones are coming up in the next 2 weeks based on the timeline?",
   "MUST reference actual milestones from project_timeline.csv; for the configured reporting date (2026-04-22) these are the Phase 2 deliverables 'Current systems audit report' and 'Gap analysis report'. MUST NOT invent milestones."),
  ("T12", "Generate a stakeholder update email summarizing Sprint 2 progress.",
   "MUST be a professional email referencing Sprint 2 tasks (T004 and T005 done, T006 in progress). MUST NOT include Sprint 3 or later tasks."),
]
print(len(TESTS), "tests")
```

```
12 tests
```

3 · Run Mira on every test

[12] ✓ 1m

```
def call_mira(test_id, prompt, retries=2):
    for attempt in range(retries + 1):
        try:
            t = time.time()
            r = requests.post(MIRA_WEBHOOK_URL, json={"chatInput": prompt, "sessionId": f"deepeval-{test_id}", "test_id": test_id}, timeout=180)
            r.raise_for_status()
            data = r.json()
            data = data[0] if isinstance(data, list) else data
            data["client_latency_s"] = round(time.time() - t, 2)
            return data
        except Exception as e:
            if attempt == retries:
                print(f"  {test_id}: FAILED — {e}")
                return {"output": "", "route": "ERROR", "error": str(e)}
            time.sleep(3)

responses = {}
for tid, prompt, _ in TESTS:
    responses[tid] = call_mira(tid, prompt)
    r = responses[tid]
    print(f"{tid:4} route={r.get('route'):11} latency={r.get('client_latency_s')}s  trace={r.get('trace_id')}")
```

```
T1   route=PLAN        latency=27.37s  trace=94b4a79c69407c900ebde57ae21a3e50
T2   route=CLARIFY     latency=2.85s  trace=01552aaf1af7674118cc3c3b0b75953d
T3   route=RISK        latency=19.24s  trace=880cfff6edd52ee8932c8d3fe390e068
T4   route=CLARIFY     latency=2.4s  trace=8d86c8efa720b45dbce56e4a827d34a3
T5   route=STATUS      latency=7.69s  trace=993a5524456041450dfb6ded35513e2
T6   route=CLARIFY     latency=2.63s  trace=39c387187b2b676217ec2fce7516a2f4
T7   route=RISK        latency=6.71s  trace=0ee2cfeffa5b3c17ecc02b7d1704740c
T8   route=STATUS      latency=6.03s  trace=af17c08f44b144b2cb1db5a17f0ebbf7
T9   route=CLARIFY     latency=2.73s  trace=824013d0947eb9295cb144bd399c24d9
T10  route=STATUS      latency=3.68s  trace=ed23a9572d09a2c8e3e06c42de0deafd
T11  route=MILESTONES  latency=4.99s  trace=62e4ac444c8d79319f3fc9ed19660271
T12  route=COMMS       latency=6.72s  trace=c0103b1f4c3c9dbb45e1558bc442cd4d
```

4 · Build DeepEval test cases

[15] ✓ 4s

```
from deepeval.test_case import LLMTestCase

def chunk_context(text, max_chars=1500):
    """Split Mira's fact pack into section-sized chunks for retrieval_context."""
    if not text:
        return None
    chunks, cur = [], ""
    for line in text.splitlines():
        if len(cur) + len(line) > max_chars and cur:
            chunks.append(cur.strip()); cur = ""
        cur += line + "\n"
    if cur.strip():
        chunks.append(cur.strip())
    return chunks

cases = {}
for tid, prompt, expected in TESTS:
    r = responses[tid]
    cases[tid] = LLMTestCase(
        input=prompt,
```

```
            actual_output=r.get("output") or "(no output)",
            expected_output=expected,
            retrieval_context=chunk_context(r.get("retrieval_context")),
            name=tid,
        )
print("Cases with retrieval context (faithfulness applies):", [t for t, c in cases.items() if c.retrieval_context])
```

```
Cases with retrieval context (faithfulness applies): ['T1', 'T3', 'T5', 'T7', 'T8', 'T10', 'T11', 'T12']
```

5 · Score with DeepEval

```
from deepeval.metrics import FaithfulnessMetric, AnswerRelevancyMetric, GEval
from deepeval.test_case import SingleTurnParams   # DeepEval 4.x name (was LLMTestCaseParams in 3.x)

def make_metrics():
    return {
        "faithfulness": FaithfulnessMetric(threshold=THRESHOLD, model=JUDGE_MODEL, include_reason=True, async_mode=False),
        "answer_relevancy": AnswerRelevancyMetric(threshold=THRESHOLD, model=JUDGE_MODEL, include_reason=True, async_mode=False),
        "requirement_compliance": GEval(
            name="Requirement Compliance",
            evaluation_steps=[
                "Read the expected output: it is a rubric of MUST and MUST NOT requirements, not text to match.",
                "Check whether the actual output satisfies each MUST requirement for the given input.",
                "Check whether the actual output violates any MUST NOT requirement; any violation should heavily reduce the score.",
                "Ignore formatting, headings, footers, citations and wording differences; judge substance only.",
                "Give a high score only if every requirement is met.",
            ],
            evaluation_params=[SingleTurnParams.INPUT, SingleTurnParams.ACTUAL_OUTPUT, SingleTurnParams.EXPECTED_OUTPUT],
            model=JUDGE_MODEL, threshold=THRESHOLD, async_mode=False),
    }

rows, judge_cost = [], 0.0
for tid, case in cases.items():
    row = {"test_id": tid, "route": responses[tid].get("route"), "trace_id": responses[tid].get("trace_id"),
           "mira_latency_ms": responses[tid].get("latency_ms"), "mira_est_cost_usd": responses[tid].get("est_cost_usd")}
    for name, metric in make_metrics().items():
        if name == "faithfulness" and not case.retrieval_context:
            row[name], row[f"{name}_reason"] = None, "n/a — clarification (no facts used)"
            continue
        try:
            metric.measure(case)
            row[name], row[f"{name}_reason"] = round(metric.score, 3), metric.reason
            judge_cost += metric.evaluation_cost or 0
        except Exception as e:
            row[name], row[f"{name}_reason"] = None, f"error: {e}"
    scored = [row[m] for m in ("faithfulness", "answer_relevancy", "requirement_compliance") if row[m] is not None]
    row["all_pass"] = all(s >= THRESHOLD for s in scored) if scored else False
    rows.append(row)
    print(f"{tid:4} faith={row['faithfulness']}  relevancy={row['answer_relevancy']}  compliance={row['requirement_compliance']}")

df = pd.DataFrame(rows)
print(f"\nJudge cost for this run: ${judge_cost:.4f}")
```

```
T1   faith=1.0  relevancy=0.895  compliance=0.643
T2   faith=None  relevancy=0.909  compliance=0.947
T3   faith=1.0  relevancy=1.0  compliance=0.766
T4   faith=None  relevancy=0.7  compliance=1.0
T5   faith=0.882  relevancy=0.909  compliance=0.709
T6   faith=None  relevancy=0.636  compliance=0.986
T7   faith=0.714  relevancy=0.889  compliance=1.0
T8   faith=1.0  relevancy=0.412  compliance=0.846
T9   faith=None  relevancy=0.2  compliance=0.846
T10  faith=1.0  relevancy=0.5  compliance=1.0
T11  faith=1.0  relevancy=0.571  compliance=0.979
T12  faith=0.889  relevancy=0.875  compliance=0.976

Judge cost for this run: $0.0208
```

6 · Results

```
summary = pd.DataFrame({
    "metric": ["faithfulness", "answer_relevancy", "requirement_compliance"],
    "mean_score": [df[m].mean() for m in ("faithfulness", "answer_relevancy", "requirement_compliance")],
    "pass_rate": [f"{(df[m].dropna() >= THRESHOLD).sum()}/{df[m].notna().sum()}" for m in ("faithfulness", "answer_relevancy", "requirement_compliance")],
}).round(3)
display(summary)
print(f"Tests passing all applicable metrics: {df['all_pass'].sum()}/{len(df)}")
print(f"Mira avg latency: {df['mira_latency_ms'].mean()/1000:.1f}s | Mira est. cost for 12 requests: ${df['mira_est_cost_usd'].sum():.4f}")
display(df[["test_id", "route", "faithfulness", "answer_relevancy", "requirement_compliance", "all_pass"]])
```

	metric	mean_score	pass_rate
0	faithfulness	0.936	8/8
1	answer_relevancy	0.708	7/12
2	requirement_compliance	0.892	11/12

```
Tests passing all applicable metrics: 6/12
Mira avg latency: 7.0s | Mira est. cost for 12 requests: $0.0078
```

	test_id	route	faithfulness	answer_relevancy	requirement_compliance	all_pass
0	T1	PLAN	1.000	0.895	0.643	False
1	T2	CLARIFY	NaN	0.909	0.947	True
2	T3	RISK	1.000	1.000	0.766	True
3	T4	CLARIFY	NaN	0.700	1.000	True
4	T5	STATUS	0.882	0.909	0.709	True
5	T6	CLARIFY	NaN	0.636	0.986	False
6	T7	RISK	0.714	0.889	1.000	True
7	T8	STATUS	1.000	0.412	0.846	False
8	T9	CLARIFY	NaN	0.200	0.846	False
9	T10	STATUS	1.000	0.500	1.000	False
10	T11	MILESTONES	1.000	0.571	0.979	False
11	T12	COMMS	0.889	0.875	0.976	True

Next steps: Generate code with summary | New interactive sheet

```
# Reasons for anything below threshold — read these before trusting a score
for _, r in df.iterrows():
    for m in ("faithfulness", "answer_relevancy", "requirement_compliance"):
        if r[m] is not None and r[m] == r[m] and r[m] < THRESHOLD:
            print(f"{r['test_id']} · {m} = {r[m]}\n   {r[f'{m}_reason']}\n")
```

```
T1 · requirement_compliance = 0.643
   The response includes all required phases, milestones, and a timeline that aligns with the project description. However, it violates the MUST NOT requirement by including open items related to budget and KPIs that were not specified in the input, which detracts from the overall alignment with the expected

T6 · answer_relevancy = 0.636
   The score is 0.64 because several statements in the output were irrelevant to generating the status report, such as mentioning a lack of data and discussing risks and milestones instead of focusing on the current status. These irrelevant points detracted from the overall relevance of the response.

T8 · answer_relevancy = 0.412
   The score is 0.41 because many statements in the output did not directly address the question about tasks being blocked or at risk of missing deadlines. Instead, they provided irrelevant details about task ownership and unrelated activities, which detracted from the main focus.

T9 · answer_relevancy = 0.2
   The score is 0.20 because the output included numerous irrelevant statements that did not directly address the request for a project plan, such as emphasizing the need for details and elements that were not provided. While these points are important in a broader context, they detracted from the specific t

T10 · answer_relevancy = 0.5
   The score is 0.50 because while some relevant information was provided, many statements about blocked and overdue tasks did not directly address the specific request for a summary of tasks done, in progress, and to do. This led to a lack of clarity in the response.

T11 · answer_relevancy = 0.571
   The score is 0.57 because the output included irrelevant statements about reports and use cases scheduled beyond the next 2 weeks, which did not address the specific request for upcoming milestones within the specified timeframe.
```

```
df.to_csv("mira_deepeval_results.csv", index=False)
with open("mira_deepeval_responses.json", "w") as f:
    json.dump(responses, f, indent=2)
try:
    from google.colab import files
    files.download("mira_deepeval_results.csv")
except Exception:
    print("Saved mira_deepeval_results.csv and mira_deepeval_responses.json")
```

7 · Attach scores to Langfuse traces

```
def push_score(trace_id, name, value, comment):
    body = {"id": f"{trace_id}-deepeval-{name}", "traceId": trace_id, "name": f"deepeval_{name}",
            "value": float(value), "dataType": "NUMERIC", "comment": (comment or "")[:1000]}
    r = requests.post(f"{LANGFUSE_HOST}/api/public/scores", json=body, auth=LF_AUTH, timeout=30)
    return r.status_code

if PUSH_TO_LANGFUSE:
    sent, failed = 0, []
    for _, r in df.iterrows():
        if not r["trace_id"]:
            continue
        for m in ("faithfulness", "answer_relevancy", "requirement_compliance"):
            if r[m] is not None and r[m] == r[m]:
                code_ = push_score(r["trace_id"], m, r[m], r[f"{m}_reason"])
                sent += code_ < 300
                if code_ >= 300: failed.append((r["test_id"], m, code_))
    print(f"Sent {sent} scores to Langfuse.", f"Failed: {failed}" if failed else "")
    print("Langfuse → Tracing → filter tag = test ID (e.g. T5) → Scores tab")
```

```
Sent 29 scores to Langfuse. Failed: [('T12', 'faithfulness', 429), ('T12', 'answer_relevancy', 429), ('T12', 'requirement_compliance', 429)]
```

```
Langfuse → Tracing → filter tag = test ID (e.g. T5) → Scores tab
```

Colab paid products - Cancel Google AI contracts here